AI-Based Stock Portfolio Management with Neural Time-Series Forecasting and Meta-Labeling

Mehmet Emre Özfatura   Alanur Ersoy   Ege Serin   Görkem Güller
Sabancı University, Istanbul, Turkey

May 21, 2026

Abstract

Accurate stock price forecasting remains a fundamental challenge in quantitative finance due to the highly volatile, noisy and non-linear nature of financial markets. This paper presents an AI-based stock portfolio management system that combines neural time-series forecasting with a meta-labeling pipeline for robust trading decision making. We investigate three forecasting architectures: TimesNet, TimeMixer and LightGBM. Models evaluated across a portfolio of large-cap technology equities including Apple (AAPL), Nvidia (NVDA), Google (GOOGL), Microsoft (MSFT) and Meta (META), using real daily market data retrieved from Polygon.io. A LightGBM meta-classifier is applied on top of the primary forecasters to filter low-confidence trading signals, converting raw price predictions into high-precision portfolio decisions. By evaluating the system across multiple correlated equities operating in the same sector, we assess both per-asset forecasting accuracy and the generalisability of the meta-labeling pipeline under varying market regimes.

1 Introduction

Financial markets are highly complex systems characterised by volatility, non-linearity and rapid regime changes that make accurate price forecasting extremely difficult. Traditional fundamental and technical analysis methods are often insufficient for capturing these dynamics, motivating the adoption of deep learning approaches for market modelling [Lopez de Prado, 2018].

The goal of this work is to develop an AI-based stock portfolio management system that (i) learns temporal patterns from raw market data using neural time-series models and (ii) converts raw price predictions into high-confidence trading signals through a meta-labeling framework. Rather than evaluating on a single equity, we focus on a portfolio of major large-cap technology companies (AAPL, NVDA, GOOGL, MSFT and META) which share sector-level dynamics while exhibiting distinct price behaviors and volatility profiles. This multi-asset setting provides a more realistic and challenging benchmark for evaluating both forecasting accuracy and the robustness of the decision pipeline.

We adopt a two-stage pipeline. In the first stage, three forecasting models are compared: TimesNet [Wu et al., 2023], which exploits multi-periodicity via frequency-domain analysis; TimeMixer [Wang et al., 2024], a lightweight MLP-based architecture using multiscale decomposition; and LightGBM [Ke et al., 2017] as a gradient boosting baseline operating on hand-crafted lag features. In the second stage, a separate LightGBM meta-classifier evaluates each forecasted signal against the Triple Barrier Method to produce binary profitability labels, filtering out low-confidence trades before execution.

The paper is organised as follows. Section **??** reviews related work on time-series forecasting, neural architectures and meta-labeling. Section **??** describes the system architecture, model formulations and the meta-labeling pipeline. Section 4 details the experimental setup including the dataset, preprocessing, and evaluation protocol. Section 5 presents and analyses results across all assets. Section **??** concludes with a summary of findings and directions for future work.

Fan and Shen [2024], Li et al. [2025], Li et al. [2024]

1.1 Meta-Labeling

The meta-labeling framework introduced by Lopez de Prado [2018] separates the concerns of signal generation and bet sizing. A primary model generates directional forecasts; a secondary model is then trained to predict whether each primary signal will be profitable, acting as a filter. This two-stage design allows the secondary model to focus on a binary decision (trade or skip) rather than price-level regression, which is a significantly easier task. Profitability labels are typically derived from the Triple Barrier Method, which marks each signal as a win or loss depending on whether the price first reaches a profit-take or stop-loss barrier within a fixed time window [Lopez de Prado, 2018].

2 Preliminaries and problem formulation

2.1 Time series forecasting

Let $\mathcal{X} = \{X^1, X^2, \ldots, X^N\}$ denote a portfolio of N stocks, where each $X^i \in \mathbb{R}^{T\times C}$ represents the full observed history of asset i: T is the total number of trading days and C is the number of recorded features per day. The forecasting task is to predict the next H steps from the most recent lookback window of length L:

$$\hat{X}^i_{t+1:t+H} = f_\theta\big(X^i_{t-L+1:t}\big)\,, \qquad \hat{X}^i_{t+1:t+H} \in \mathbb{R}^{H\times C}, \tag{1}$$

where $f_\theta(\cdot)$ is a parameterised forecasting function with learnable parameters θ. All models are trained independently per asset to avoid cross-contamination of learned representations. Performance is reported both per asset and as a macro-average across the portfolio.

2.2 Neural time-series forecasting

2.2.1 TimesNet

TimesNet [Wu et al., 2023] models time series by exploiting multi-periodicity through frequency-domain analysis. Given an input sequence $\mathbf{X} \in \mathbb{R}^{L\times C}$, where L denotes the lookback window length and C the number of input features, the Fast Fourier Transform (FFT) identifies dominant frequencies and their corresponding periods $\{p_1, \ldots, p_k\}$. The 1D sequence is then reshaped into multiple 2D temporal representations:

$$\mathbf{X}^{(i)}_{\text{2D}} = \text{Reshape}_{p_i}(\mathbf{X}), \quad i = 1, \ldots, k. \tag{2}$$

Each 2D representation is processed using parameter-efficient Inception-style convolutional kernels inside stacked *TimesBlock* modules. The resulting outputs are mapped back to 1D space and adaptively aggregated to produce the block output. TimesNet captures both intra-period (within-cycle) and inter-period (cross-cycle) variations, making it particularly suited to financial data where weekly, monthly and quarterly periodicities co-exist.

2.2.2 TimeMixer

TimeMixer [Wang et al., 2024] adopts a pure MLP-based architecture that explicitly avoids attention mechanisms. Its core design principle is multiscale temporal mixing. The model first constructs multiscale representations through average downsampling:

$$\mathbf{X}_m \in \mathbb{R}^{\lfloor L/2^m \rfloor \times C}, \quad m = 0, \ldots, M, \tag{3}$$

capturing temporal dynamics at increasingly coarse resolutions. These representations are then processed by *Past-Decomposable Mixing* (PDM) blocks, which decompose historical information into seasonal and trend components and enable bidirectional information flow across temporal scales:

$$\mathbf{X}^{(l)} = \text{PDM}\Big(\mathbf{X}^{(l-1)}\Big)\,. \tag{4}$$

TimeMixer was selected for its computational efficiency and architectural simplicity, which makes it an attractive option when training over multiple assets with limited data per equity.

2.2.3 LightGBM

LightGBM [Ke et al., 2017] is a gradient boosting decision tree (GBDT) framework that achieves high computational efficiency through Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB). At each node, the best split is found by maximising the variance gain:

$$V_j(d) = \frac{1}{|\mathcal{O}|}\left[\frac{\left(\sum_{i\in\mathcal{O}_l} g_i\right)^2}{n_l^j(d)} + \frac{\left(\sum_{i\in\mathcal{O}_r} g_i\right)^2}{n_r^j(d)}\right], \tag{5}$$

where g_i is the residual gradient of instance i and $n_l^j(d)$, $n_r^j(d)$ are the instance counts on each side of split threshold d.

In the forecasting stage, lag features from the lookback window $\mathbf{X}_{(t-L:t,\,\cdot)} \in \mathbb{R}^{L\times C}$ are used as input and each horizon step $h \in \{1,\dots,H\}$ is treated as a separate regression target. Crucially, predictions are made in *delta space*: the model predicts price changes relative to the last known closing price and absolute prices are recovered by adding the predicted delta to the last observation. This transformation aligns the prediction target's scale with the feature space and substantially improves accuracy compared to direct absolute-price prediction.

2.3 Stock price forecasting

Given set of N stocks $\mathcal{X} = \{X^1, X^2, \dots, X^N\}$, each stock $X^i \in \mathbb{R}^{T\times C}$ represents the historical data of lookback window length T, and C recorded features or each time instant t. Following the previous works Fan and Shen [2024], Li et al. [2024], Wang et al. [2025], our objective is to predict closing price $p_{close}^{(t+d,i)}$ on trading day t and calculate the d-day return ratio

$$r_{t+d}^i = \frac{p_{close}^{(t+d,i)} - p_{close}^{(t,i)}}{p_{close}^{(t,i)}}, \tag{6}$$

and we also interested rolling average d-day return ratio

$$\bar{r}_{t+d}^i = \frac{\bar{p}_{close}^{(t+d,i)} - p_{close}^{(t,i)}}{p_{close}^{(t,i)}}, \tag{7}$$

where $p_{close}^{(t+d,i)}$ is the rolling average closing price of the stock

$$\bar{p}_{close}^{(t+d,i)} = \sum_{j=0}^{L-1} w_j\, p_{close}^{(t+d-j,i)} \tag{8}$$

where the w_j is the weight of the $t+d-j$th day when computing the rolling average for daily closed price.

2.3.1 Performance Metrics

Following the previous works Fan and Shen [2024], Wang et al. [2025], Hu et al. [2025], we briefly summarized the common performance metrics used for evaluation of the forecasting methods:

Information Coefficient (IC)

The Information Coefficient (IC) measures the linear correlation between a model's predicted scores $\hat{y}$ and the realized returns y across N assets in a given cross-section. Given predicted scores $\hat{y}_i$ and realized returns y_i for $i = 1,\dots,N$ assets, the IC is defined as the *Pearson correlation coefficient*:

$$\mathrm{IC} = \rho(\hat{y},\, y) = \frac{\sum_{i=1}^N (\hat{y}_i - \bar{\hat{y}})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^N (\hat{y}_i - \bar{\hat{y}})^2}\cdot\sqrt{\sum_{i=1}^N (y_i - \bar{y})^2}}, \tag{9}$$

where $\bar{\hat{y}}$ and $\bar{y}$ are the cross-sectional means of predictions and returns, respectively. IC $\in [-1, 1]$; higher values indicate stronger agreement between predictions and outcomes.

Rank Information Coefficient (RIC)
It measures the *monotonic* (rank-order) agreement between predicted scores and realized returns using the *Spearman correlation coefficient*. Let $r_i = \mathrm{rank}(\hat{y}_i)$ and $s_i = \mathrm{rank}(y_i)$ denote the rank of asset i in predictions and realized returns, respectively. The RIC is:

$$\mathrm{RIC} = \rho_s(\hat{y},\, y) = 1 - \frac{6\sum_{i=1}^{N} d_i^2}{N(N^2-1)}, \tag{10}$$

where $d_i = r_i - s_i$ is the rank difference for asset i. Equivalently (and exactly, without ties):

$$\mathrm{RIC} = \frac{\sum_{i=1}^{N}(r_i-\bar{r})(s_i-\bar{s})}{\sqrt{\sum_{i=1}^{N}(r_i-\bar{r})^2}\cdot\sqrt{\sum_{i=1}^{N}(s_i-\bar{s})^2}}. \tag{11}$$

3 Meta-Labeling Pipeline

Following Lopez de Prado [2018], trading labels are generated using the Triple Barrier Method. For each forecasted signal at time t, symmetric horizontal barriers (profit-take and stop-loss at $\pm\delta$) and a vertical barrier (maximum holding period τ) are placed. A binary label $y_t \in \{0, 1\}$ is assigned:

$$y_t = \begin{cases} 1 & \text{if the profit-take barrier is touched first,} \\ 0 & \text{if the stop-loss or vertical barrier is touched first.} \end{cases} \tag{12}$$

A LightGBM meta-classifier is then trained on these labels using features derived from both the primary model's outputs and market context: predicted return magnitude, technical indicators (MACD histogram, RSI, Bollinger Band width) and market microstructure statistics (transaction count, VWAP deviation). At inference time, signals classified as unprofitable ($\hat{y}_t = 0$) are filtered out before execution. This pipeline is applied independently per asset, allowing the meta-classifier to learn asset-specific trade filtering patterns.

4 Experiments

4.1 Dataset and Preprocessing

All experiments use daily price data for five large-cap technology equities (AAPL, NVDA, GOOGL, MSFT and META) which are retrieved from a proprietary minute-bar dataset provided by Polygon.io covering 2021–2025. The raw minute-bar data (≈1.8 million records per asset) is resampled to daily resolution using standard market aggregation rules: Open is the first bar's open, High is the intraday maximum, Low is the intraday minimum, Close is the last bar's close and Volume is the daily sum. VWAP (volume-weighted average price) and transaction count are retained as supplementary features, yielding $C = 7$ input features per asset per trading day.

Data prior to 2022 is excluded to avoid pandemic-related market anomalies, leaving approximately 939 trading days per asset after removing non-trading days. All assets share the same trading calendar, so the chronological splits are perfectly aligned across the portfolio. Data is split into 70% training, 15% validation and 15% test sets with no shuffling to preserve temporal ordering. A sliding window approach is used with a lookback window of $L = 14$ days and a forecast horizon of $H = 5$ days.

4.2 Model Configurations

The architecture-specific parameters for each model are summarised in Tables 1, 2, and 3. All hyperparameters are held constant across assets to enable fair cross-asset comparison; no asset-specific tuning is performed.

4.3 Evaluation Protocol

Forecasting performance is measured using Mean Squared Error (MSE), Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) on the held-out test split. Per-target breakdowns are reported for the High and Close price

Table 1: TimesNet architecture parameters.

Parameter	**Value**	**Description**
`e_layers`	2	Number of stacked TimesBlock layers
`top_k`	3	Dominant periods selected per block
`d_model`	32	Token embedding dimension
`d_ff`	64	Feed-forward hidden dimension
`num_kernels`	6	Inception branch count
`dropout`	0.1	Dropout probability

Table 2: TimeMixer architecture parameters.

Parameter	**Value**	**Description**
`pdm_blocks`	2	Number of PDM layers
`downsampling_layers`	1	Downsampling steps
`downsampling_window`	2	Pooling window size
`scales` (M)	2	Multiscale inputs: [14, 7]
`d_model`	64	Embedding dimension
`d_ff`	128	Feed-forward hidden dimension
`dropout`	0.1	Dropout probability

channels, as these are most directly relevant to portfolio decisions. Results are reported per asset and as a macro-average across the portfolio.

For the meta-labeling stage, signal filtering quality is assessed via Precision, Recall and F1 Score relative to the unfiltered baseline. Strategy-level performance is measured using the annualised Sharpe Ratio and the Probabilistic Sharpe Ratio (PSR) [Lopez de Prado and Bailey, 2014], which corrects for non-normality and selection bias in backtested returns. Return distribution quality is also examined through skewness and excess kurtosis.

5 Results

6 Discussion

6.1 volatility normalization

Lee et al. [2025], Lin et al. [2025]

Acknowledgments

This work was carried out as part of the ENS 491/492 Graduation Project at Sabancı University under the supervision of Mehmet Emre Özfatura. Data access was provided via Polygon.io.

Table 3: LightGBM parameters for the price forecaster and meta-classifier.

Parameter	**Forecaster**	**Meta-Classifier**
`objective`	regression	binary
`metric`	mae	binary_logloss
`n_estimators`	1000	500
`num_leaves`	31	31
`learning_rate`	0.05	0.05
`min_child_samples`	10	20
`is_unbalance`	—	True
`n_features`	31	11

REFERENCES

Jinyong Fan and Yanyan Shen. Stockmixer: a simple yet strong mlp-based architecture for stock price forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 8389–8397, 2024.

Yifan Hu, Yuante Li, Peiyuan Liu, Yuxia Zhu, Naiqi Li, Tao Dai, Shu-tao Xia, Dawei Cheng, and Changjun Jiang. Fintsb: A comprehensive and practical benchmark for financial time series forecasting. *arXiv preprint arXiv:2502.18834*, 2025.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL `https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html`.

Hyunwoo Lee, Jihyeong Jeon, Jaemin Hong, and U Kang. Mitigating distribution shift in stock price data via return-volatility normalization for accurate prediction. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, CIKM '25, page 1458–1467, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400720406. URL `https://doi.org/10.1145/3746252.3761067`.

Shuozhe Li, Zachery B Schulwol, and Risto Miikkulainen. Transformer based time-series forecasting for stock. *arXiv preprint arXiv:2502.09625*, 2025.

Tong Li, Zhaoyang Liu, Yanyan Shen, Xue Wang, Haokun Chen, and Sen Huang. Master: Market-guided stock transformer for stock price forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 162–170, 2024.

Sida Lin, Yankai Chen, Yiyan Qi, Chenhao Ma, Bokai Cao, Yifei Zhang, Xue Liu, and Jian Guo. Cspo: Cross-market synergistic stock price movement forecasting with pseudo-volatility optimization. In *Companion Proceedings of the ACM on Web Conference 2025*, WWW '25, page 354–363, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400713316. URL `https://doi.org/10.1145/3701716.3715216`.

Marcos Lopez de Prado. *Advances in Financial Machine Learning*. John Wiley & Sons, 2018.

Marcos Lopez de Prado and David H. Bailey. The deflated sharpe ratio: Correcting for selection bias, backtest overfitting and non-normality. *Journal of Portfolio Management*, 40(5):94–107, 2014. URL `https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2460551`.

Mengyu Wang, Tiejun Ma, and Shay B. Cohen. Pre-training time series models with stock data customization. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, KDD '25, page 3019–3030, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400714542. doi: 10.1145/3711896.3737005. URL `https://doi.org/10.1145/3711896.3737005`.

Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Y. Zhang, and Jun Zhou. TimeMixer: Decomposable multiscale mixing for time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2024. URL `https://openreview.net/forum?id=7oLshfEIC2`.

Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. TimesNet: Temporal 2d-variation modeling for general time series analysis. In *International Conference on Learning Representations (ICLR)*, 2023. URL `https://openreview.net/forum?id=ju_Uqw384Oq`.

A NOTATION SUMMARY

Table 4: Global notation used throughout this paper.

Notation	**Definition**
$\mathbf{X}^{(s)} \in \mathbb{R}^{T \times C}$	Observed time series for asset s
$\mathcal{S}$	Set of assets: $\{$AAPL, NVDA, GOOGL, MSFT, META$\}$
T	Total length of the observed time series
H	Prediction horizon
C	Number of variables (channels)
L	Lookback window length
$f_\theta(\cdot)$	Parameterised forecasting function
θ	Learnable model parameters
p_i	i-th dominant period identified via FFT (TimesNet)
k	Number of selected dominant periods (TimesNet)
m	Scale index (TimeMixer)
M	Maximum number of scales (TimeMixer)
g_i	Residual gradient of instance i (LightGBM)
y_t	Binary profitability label from Triple Barrier Method
δ	Barrier width for profit-take / stop-loss
τ	Maximum holding period (vertical barrier)